

Introduction to Natural Language Processing

Naeemullah Khan

naeemullah.khan@kaust.edu.sa

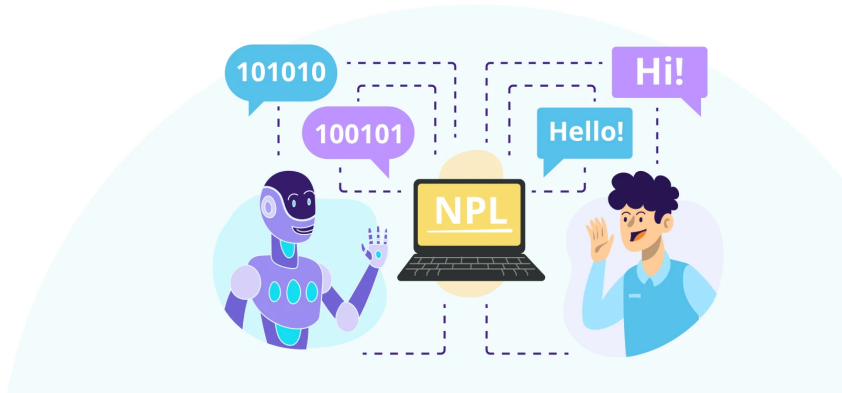


جامعة الملك عبد الله
للعلوم والتقنية
King Abdullah University of
Science and Technology

KAUST Academy
King Abdullah University of Science and Technology

July 31, 2026

Natural Language Processing



Source: Byteridge, "Natural Language Processing Business Use Cases".

1. Motivation
2. Learning Outcomes
3. Introduction
4. Classical Text Classification
5. N-grams
6. Sequence Notation
7. Probabilistic Notation
8. Probability of a Sequence
9. Start and End Tokens
10. Count and Probability Matrices
11. Out of vocabulary words
12. Smoothing
13. Evaluating Language Models: Perplexity

14. Limitations

15. Summary

16. References

- ▶ Language helps us talk to each other, and now to computers too.
- ▶ NLP (Natural Language Processing) teaches machines to understand, interpret, and generate human language.
- ▶ Why is NLP important?
 - Makes **chatbots** like ChatGPT and Siri possible
 - Powers **search engines** like Google
 - Helps **translate languages** (Google Translate)
 - Finds out what **people feel in reviews** (sentiment analysis)
- ▶ NLP is foundational to advanced AI systems.

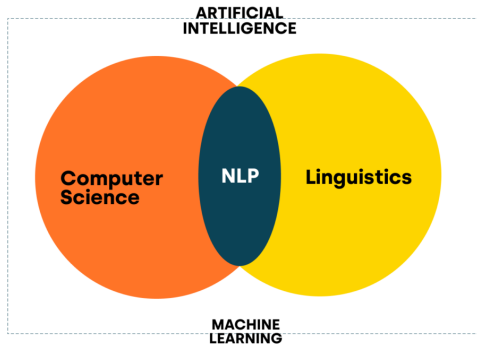
- ▶ **Define** Natural Language Processing (NLP) and describe its core tasks
- ▶ **Apply** classical text classification with Naive Bayes and logistic regression
- ▶ **Explain and construct** N-grams (unigram, bigram, trigram) using sequence notation
- ▶ **Compute** N-gram probabilities, count matrices, and sequence probabilities
- ▶ **Apply** start/end tokens and handle unknown words (OOV, UNK)
- ▶ **Smooth** N-gram estimates so unseen contexts receive non-zero probability
- ▶ **Evaluate** a language model with perplexity
- ▶ **Recognize** the limitations of N-gram models and future directions

Natural Language Processing: **Introduction**

What is NLP?

- ▶ Study of computational approaches to processing natural languages.
- ▶ Processing includes:
 - Acquiring language data
 - Representing information
 - Storing text and speech
 - Understanding meaning
 - Characterizing language patterns
 - Generating new language
- ▶ Natural languages refer to human languages.

What is Natural Language Processing?



Source: Qtravel.ai, "What is Natural Language Processing".

NLP = Computer Science + Linguistics + AI

► Deals with:

- Language understanding (input)
- Language generation (output)
- Language translation
- Information extraction

► Subfields:

- Syntax
- Semantics
- Pragmatics
- Discourse



Goal: Deep Understanding

Requires context, linguistic structure, meanings...



To Avoid: Shallow Matching

Could be useful also though depending on use case

Cartoon: Gary Larson, *The Far Side*.

Goal of NLP:

- ▶ Enable machines to understand and generate human language.
- ▶ Facilitate human-computer interaction through natural language.
- ▶ Develop systems that can process and analyze large amounts of text data.

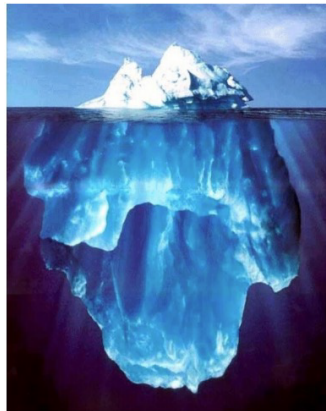
- ▶ **Text Preprocessing**
 - Cleaning and preparing raw text for analysis.
- ▶ **Tokenization**
 - Splitting text into words, sentences, or other meaningful units.
- ▶ **POS Tagging**
 - Assigning parts of speech (noun, verb, etc.) to each token.
- ▶ **Parsing**
 - Analyzing grammatical structure of sentences.
- ▶ **Named Entity Recognition (NER)**
 - Identifying entities such as people, organizations, locations.
- ▶ **Sentiment Analysis / Classification**
 - Determining sentiment or categorizing text.
- ▶ **Language Modeling**
 - Predicting the next word or sequence in text.

- ▶ An iceberg is a large piece of freshwater ice that has broken off from a snow-formed glacier or ice shelf and is floating in open water.



Source: Dan Klein, UC Berkeley NLP slides (via T. Berg-Kirkpatrick, CMU).

- ▶ An iceberg is a large piece of freshwater ice that has broken off from a snow-formed glacier or ice shelf and is floating in open water.

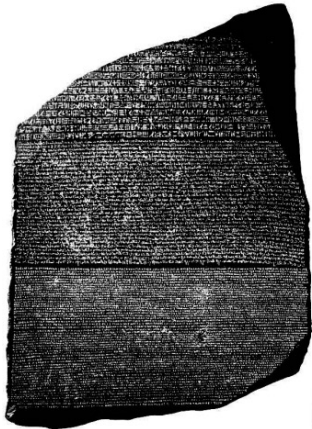


Source: Dan Klein, UC Berkeley NLP slides.

Task	What It Does	Example
Tokenization	Split text into words	"I love NLP" → ["I", "love", "NLP"]
POS Tagging	Label grammar tags	"Dogs bark" → [Noun, Verb]
Named Entity Recognition	Find names, places, etc.	"Christopher Nolan lives in Los Angeles" → [Person, Location]
Sentiment Analysis	Detect mood	"This movie was amazing!" → Positive
Machine Translation	Language to language	English → French

Applications such as summarization and question answering chain several of these tasks together.

- ▶ **Lowercase everything:** "NLP" → "nlp"
- ▶ **Removing stop words:** drop words that carry little meaning on their own, such as "is", "the" and "and".
- ▶ **Stemming / Lemmatization:** reduce a word to its root or base form, "running" → "run"
- ▶ **Language detection** (optional): identify each document's language before applying language-specific steps.
- ▶ **Spellcheck** (optional): normalize typos so variants map to the same token.
- ▶ **Vectorization:** turn the surviving tokens into numbers a model can consume.



The Rosetta Stone, British Museum: one text, three parallel scripts. Source: Dan Klein, UC Berkeley NLP slides.

What is a corpus?

- ▶ A corpus is a collection of text.
- ▶ Often annotated in some way.
- ▶ Sometimes just lots of text.
- ▶ **Balanced corpora:** Usually not possible in practice.
- ▶ **Examples:**
 - Newswire collections: 500M+ words
 - Brown corpus: 1M words of tagged “balanced” text
 - Penn Treebank: 1M words of parsed WSJ
 - Canadian Hansards: 10M+ words of aligned French/English sentences
 - The Web: billions of words of who knows what

- ▶ **Vocabulary** = All unique words in your dataset
- ▶ Example: "I love NLP and NLP loves me" →
Vocabulary = {"I", "love", "NLP", "and", "loves", "me"}
- ▶ More data = Bigger vocabulary = Harder to process!

💡 Tip: Rare words may not help; common words may not mean much.

Traditional approach: One-hot encoding

- ▶ Example: "NLP" $\rightarrow [0, 0, 1, 0, 0, 0, 0, \dots]$

Problem:

- ▶ High-dimensional (thousands of words!)
- ▶ Sparse (mostly 0s)
- ▶ No meaning in structure (no relation between "king" and "queen")
- ▶ ✗ Not efficient for learning

Feature extraction = Turning text into numbers

- ▶ Count how often each word appears (**Term Frequency**)

Example:

Text	"great product"	"bad product"
Word: "great"	1	0
Word: "bad"	0	1

 Use this to find patterns in sentiment, spam, etc.

Suppose you're classifying reviews:

Positive Reviews: ["amazing", "good", "great"]

Negative Reviews: ["bad", "awful", "terrible"]

Count how often each word appears in each class.

Example table:

Word	Positive Count	Negative Count
good	20	1
bad	1	30

Helps models detect the “tone” (**sentiment clues**) of new text.

- ▶ **Bag of Words (BoW)**: Just counts word frequencies in each document.
- ▶ **TF-IDF (Term Frequency-Inverse Document Frequency)**: Adjusts for how “unique” or important a word is in a document compared to all documents.
 - Words like "the", "is" are less important.
- ▶ **Word Embeddings** (Word2Vec and successors, covered later): learn dense vectors that place related words near each other, so similarity and analogy become geometry rather than string matching.

BoW and TF-IDF both throw word order away. That is enough for topic and sentiment, but not for anything that depends on which word follows which.

NLP: Classical Text Classification

Bayes' Rule:

$$P(\text{Class} \mid \text{Words}) = \frac{P(\text{Words} \mid \text{Class}) \cdot P(\text{Class})}{P(\text{Words})}$$

- ▶ Predict class (e.g., spam or not) given the words
- ▶ Simple, powerful tool used in Naive Bayes classifiers
- ▶ Helps models reason with uncertainty

Simple, fast probabilistic model

Assumes:

- ▶ Words are independent given the class (a “naive” assumption)

Example:

- ▶ "This movie is awesome" $\rightarrow P(\text{Positive})$ high
- ▶ "This movie is boring" $\rightarrow P(\text{Negative})$ high

✓ Works well with BoW, TF-IDF, and N-grams

Logistic regression is a probability-based classifier we have already met; only the features change here.

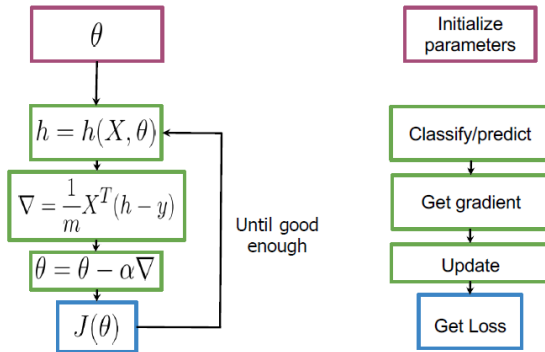
- ▶ It takes word features (from BoW or TF-IDF)
- ▶ It learns weights to predict if text belongs to a certain class, such as:
 - Positive or Negative
 - Spam or Not Spam
- ▶ Outputs a probability:

$$P(\text{Class} = 1 \mid \text{features}) = \sigma(w_1x_1 + w_2x_2 + \dots + b)$$

- ▶ Words like “awesome” get high weights for positive class

🕒 **Intuition:** each word casts a weighted “vote” toward a class, and training decides how loud each vote is.

Logistic Regression for Text Classification (cont.)



Training a Logistic Regression Model. Source: DeepLearning.AI, *NLP with Classification and Vector Spaces*, Week 1 slides.

Natural Language Processing: **N-grams**

What are N-grams?

An N-gram is a sequence of N words

Corpus: I am happy because I am learning

Unigrams: { I , am , happy , because , learning }

Bigrams: { I am , am happy , happy because ... }

✗ I happy

Trigrams: { I am happy , am happy because , ... }

Source: DeepLearning.AI, *NLP with Probabilistic Models*, Week 3 slides.

- Items are typically **words** or **characters**.

- ▶ Tokenization is the process of breaking text into smaller units called **tokens**.
- ▶ Tokens can be words, characters, or subwords.
- ▶ Example: “I love NLP” can be tokenized into:
 - Words: ["I", "love", "NLP"]
 - Characters: ["I", " ", "l", "o", "v", "e", " ", "N", "L", "P"]
- ▶ Subword tokenizers sit between the two: frequent words stay whole, rare or long ones are split into pieces the model already knows, so "tokenization" becomes ["token", "ization"].
- ▶ Every later step counts tokens, so the choice of tokenizer fixes what the model can and cannot see.

Sentence: “I love NLP”

- ▶ **Unigrams:** I, love, NLP
- ▶ **Bigrams:** I love, love NLP
- ▶ **Trigrams:** I love NLP

- ▶ Capture local word co-occurrence
- ▶ Build simple language models
- ▶ Easy to compute and analyze
- ▶ Trade-off between simplicity (unigram) and contextual richness (trigram)

Sequence Notation Basics

Sentence: $w_1, w_2, \dots, w_n$

For example: $w_1 = \text{I}$, $w_2 = \text{love}$, $w_3 = \text{NLP}$

General representation:

- ▶ Unigram: $P(w_i)$
- ▶ Bigram: $P(w_i \mid w_{i-1})$
- ▶ Trigram: $P(w_i \mid w_{i-2}, w_{i-1})$

Sequence Notation Example:

Corpus: This is great ... teacher drinks tea. $m = 500$

$w_1 \ w_2 \ w_3$ $w_{498} \ w_{499} \ w_{500}$

$$w_1^m = w_1 \ w_2 \ \dots \ w_m$$

$$w_1^3 = w_1 \ w_2 \ w_3$$

$$w_{m-2}^m = w_{m-2} \ w_{m-1} \ w_m$$

Source: DeepLearning.AI, *NLP with Probabilistic Models*, Week 3 slides.

Corpus: I am happy because I am learning

Size of corpus $m = 7$

$$P(I) = \frac{2}{7}$$

$$P(happy) = \frac{1}{7}$$

Probability of unigram:

$$P(w) = \frac{C(w)}{m}$$

Source: DeepLearning.AI, *NLP with Probabilistic Models*, Week 3 slides.

Corpus: I am happy because I am learning

$$P(am|I) = \frac{C(I \ am)}{C(I)} = \frac{2}{2} = 1$$

$$P(happy|I) = \frac{C(I \ happy)}{C(I)} = \frac{0}{2} = 0 \quad \times \text{I happy}$$

$$P(learning|am) = \frac{C(am \ learning)}{C(am)} = \frac{1}{2}$$

Probability of a bigram:
$$P(y|x) = \frac{C(x \ y)}{\sum_w C(x \ w)} = \frac{C(x \ y)}{C(x)}$$

Source: DeepLearning.AI, *NLP with Probabilistic Models*, Week 3 slides.

Corpus: I am happy because I am learning

$$P(\text{happy} | \text{I am}) = \frac{C(\text{I am happy})}{C(\text{I am})} = \frac{1}{2}$$

Probability of a trigram: $P(w_3 | w_1^2) = \frac{C(w_1^2 w_3)}{C(w_1^2)}$

$$C(w_1^2 w_3) = C(w_1 w_2 w_3) = C(w_1^3)$$

Source: DeepLearning.AI, *NLP with Probabilistic Models*, Week 3 slides.

Probability of N-gram: $P(w_N | w_1^{N-1}) = \frac{C(w_1^{N-1} w_N)}{C(w_1^{N-1})}$

$$C(w_1^{N-1} w_N) = C(w_1^N)$$

Source: DeepLearning.AI, *NLP with Probabilistic Models*, Week 3 slides.

Objective: Compute the probability of a sentence

N-gram Assumption: The probability of a word depends only on the previous $(n - 1)$ words.

Formula:

$$P(w_1^n) \approx \prod_{i=1}^n P(w_i \mid w_{i-n+1}^{i-1})$$

where w_1^n denotes the sequence $w_1, w_2, \dots, w_n$ and w_{i-n+1}^{i-1} is the context of the previous $(n - 1)$ words.

Bigram MLE:

$$P(w_i | w_{i-1}) = \frac{\text{Count}(w_{i-1}, w_i)}{\text{Count}(w_{i-1})}$$

- ▶ $\text{Count}(w_{i-1}, w_i)$: Number of times the bigram (w_{i-1}, w_i) appears in the corpus.
- ▶ $\text{Count}(w_{i-1})$: Number of times the word w_{i-1} appears as a context.

Text: “I love NLP. I love AI.”

Bigrams and Counts:

- ▶ (I, love): 2
- ▶ (love, NLP): 1
- ▶ (love, AI): 1

Probability Calculations:

- ▶ $P(\text{love} \mid \text{I}) = \frac{2}{2} = 1.0$
- ▶ $P(\text{NLP} \mid \text{love}) = \frac{1}{2} = 0.5$
- ▶ $P(\text{AI} \mid \text{love}) = \frac{1}{2} = 0.5$

- Given a sentence, what is its probability?

$$P(\textit{the teacher drinks tea}) = ?$$

- Conditional probability and chain rule reminder

$$P(B|A) = \frac{P(A, B)}{P(A)} \implies P(A, B) = P(A)P(B|A)$$

$$P(A, B, C, D) = P(A)P(B|A)P(C|A, B)P(D|A, B, C)$$

Source: DeepLearning.AI, *NLP with Probabilistic Models*, Week 3 slides.

$$P(\textit{the teacher drinks tea}) =$$

$$P(\textit{the})P(\textit{teacher}|\textit{the})P(\textit{drinks}|\textit{the teacher}) \\ P(\textit{tea}|\textit{the teacher drinks})$$

Source: DeepLearning.AI, *NLP with Probabilistic Models*, Week 3 slides.

Problem: Corpus almost never contains the exact sentence we are interested in or even its longer subsequences.

Input: the teacher drinks tea

$$P(\text{the teacher drinks tea}) = P(\text{the})P(\text{teacher}|\text{the})P(\text{drinks}|\text{the teacher})P(\text{tea}|\text{the teacher drinks})$$

$$P(\text{tea}|\text{the teacher drinks}) = \frac{C(\text{the teacher drinks tea})}{C(\text{the teacher drinks})}$$

← Both
← likely
0

Source: DeepLearning.AI, *NLP with Probabilistic Models*, Week 3 slides.

the teacher drinks tea

$$P(\text{tea}|\text{the teacher drinks}) \approx P(\text{tea}|\text{drinks})$$

$$P(\text{teacher}|\text{the})$$

$$P(\text{drinks}|\text{teacher})$$

$$P(\text{tea}|\text{drinks})$$

$$P(\text{the teacher drinks tea}) =$$

$$P(\text{the})P(\text{teacher}|\text{the})P(\text{drinks}|\text{the teacher})P(\text{tea}|\text{the teacher drinks})$$



$$P(\text{the})P(\text{teacher}|\text{the})P(\text{drinks}|\text{teacher})P(\text{tea}|\text{drinks})$$

Source: DeepLearning.AI, *NLP with Probabilistic Models*, Week 3 slides.

- Markov assumption: only last N words matter
- Bigram $P(w_n|w_1^{n-1}) \approx P(w_n|w_{n-1})$
- N-gram $P(w_n|w_1^{n-1}) \approx P(w_n|w_{n-N+1}^{n-1})$
- Entire sentence modeled with bigram $P(w_1^n) \approx \prod_{i=1}^n P(w_i|w_{i-1})$
 $P(w_1^n) \approx P(w_1)P(w_2|w_1)...P(w_n|w_{n-1})$

Source: DeepLearning.AI, *NLP with Probabilistic Models*, Week 3 slides.

Objective: Apply sequence probability approximation with bigrams.

Question:

Given these conditional probabilities

$P(\text{Mary})=0.1$; $P(\text{likes})=0.2$; $P(\text{cats})=0.3$
 $P(\text{Mary}|\text{likes})=0.2$; $P(\text{likes}|\text{Mary})=0.3$; $P(\text{cats}|\text{likes})=0.1$; $P(\text{likes}|\text{cats})=0.4$

Approximate the probability of the following sentence with bigrams: "Mary likes cats"

Type: Multiple Choice, single answer

Options and solution:

1. $P(\text{Mary likes cats}) = 0$

2. $P(\text{Mary likes cats}) = 1$

3. $P(\text{Mary likes cats}) = 0.003$

4. $P(\text{Mary likes cats}) = 0.008$

Source: DeepLearning.AI, *NLP with Probabilistic Models*, Week 3 slides.

N-gram Models: **Start and End Tokens**

- ▶ **Start:** <s> or <start>
- ▶ **End:** </s> or <end>

Benefits:

- ▶ Helps model sentence boundaries
- ▶ Enables generation and evaluation

the teacher drinks tea

$$P(\text{the teacher drinks tea}) \approx P(\text{the})P(\text{teacher}|\text{the})P(\text{drinks}|\text{teacher})P(\text{tea}|\text{drinks})$$



<s> the teacher drinks tea

$$P(\text{<s> the teacher drinks tea}) \approx P(\text{the}|\text{<s>})P(\text{teacher}|\text{the})P(\text{drinks}|\text{teacher})P(\text{tea}|\text{drinks})$$

Source: DeepLearning.AI, *NLP with Probabilistic Models*, Week 3 slides.

- Trigram:

$$P(\text{the teacher drinks tea}) \approx P(\text{the})P(\text{teacher}|\text{the})P(\text{drinks}|\text{the teacher})P(\text{tea}|\text{teacher drinks})$$

the teacher drinks tea $\Rightarrow$ <s> <s> the teacher drinks tea

$$P(w_1^n) \approx P(w_1|\text{<s> <s>})P(w_2|\text{<s> } w_1) \dots P(w_n|w_{n-2} w_{n-1})$$

- N-gram model: add N-1 start tokens <s>

Source: DeepLearning.AI, *NLP with Probabilistic Models*, Week 3 slides.

$$P(y|x) = \frac{C(x \ y)}{\sum_w C(x \ w)} = \frac{C(x \ y)}{C(x)}$$

Corpus:

<s> Lyn drinks chocolate
<s> John drinks

$$\sum_w C(drinks \ w) = 1$$

$$C(drinks) = 2$$

Source: DeepLearning.AI, *NLP with Probabilistic Models*, Week 3 slides.

End of sentence token </s> - motivation (cont.)

Corpus

<s> yes no
<s> yes yes
<s> no no
<s> no no

Sentences of length 2:

<s> yes yes
<s> yes no
<s> no no
<s> no yes

$$P(< s > \text{ yes yes}) =$$

$$P(\text{yes} \mid < s >) \times P(\text{yes} \mid \text{yes}) =$$

$$\frac{C(< s > \text{ yes})}{\sum_w C(< s > w)} \times \frac{C(\text{yes yes})}{\sum_w C(\text{yes } w)} =$$

$$\frac{2}{3} \times \frac{1}{2} = \frac{1}{3}$$

Source: DeepLearning.AI, *NLP with Probabilistic Models*, Week 3 slides.

Corpus

<s> yes no

<s> yes yes

<s> no no

Sentences of length 2:

<s> yes yes

<s> yes no

<s> no no

<s> no yes

$$P(< s > \text{ yes yes}) = \frac{1}{3}$$

$$P(< s > \text{ yes no}) = \frac{1}{3}$$

$$P(< s > \text{ no no}) = \frac{1}{3}$$

$$P(< s > \text{ no yes}) = 0$$

$$\sum_{\text{2 word}} P(\dots) = 1$$

Source: DeepLearning.AI, *NLP with Probabilistic Models*, Week 3 slides.

End of sentence token $\langle /s \rangle$ - motivation (cont.)

Corpus

$\langle s \rangle$ yes no

$\langle s \rangle$ yes yes

$\langle s \rangle$ no no

Sentences of length 3:

$\langle s \rangle$ yes yes yes

$\langle s \rangle$ yes yes no

...

$\langle s \rangle$ no no no

$$P(\langle s \rangle \text{ yes yes yes}) = \dots$$

$$P(\langle s \rangle \text{ yes yes no}) = \dots$$

$$\dots = \dots$$

$$P(\langle s \rangle \text{ no no no}) = \dots$$

$$\sum_{\text{3 word}} P(\dots) = 1$$

Source: DeepLearning.AI, *NLP with Probabilistic Models*, Week 3 slides.

Corpus

<s> yes no

<s> yes yes

<s> no no

$$\sum_{\text{2 word}} P(\dots) + \sum_{\text{3 word}} P(\dots) + \dots = 1$$

Source: DeepLearning.AI, *NLP with Probabilistic Models*, Week 3 slides.

- Bigram

$\langle s \rangle$ the teacher drinks tea $\Rightarrow$ $\langle s \rangle$ the teacher drinks tea $\langle /s \rangle$

$$P(the|\langle s \rangle)P(teacher|the)P(drinks|teacher)P(tea|drinks)P(\langle /s \rangle|tea)$$

Corpus:

$\langle s \rangle$ Lyn drinks chocolate $\langle /s \rangle$

$\langle s \rangle$ John drinks $\langle /s \rangle$

$$\sum_w C(drinks\ w) = 2$$
$$C(drinks) = 2$$

Source: DeepLearning.AI, *NLP with Probabilistic Models*, Week 3 slides.

- N-gram $\Rightarrow$ just one $\langle /s \rangle$

E.g. Trigram:

the teacher drinks tea $\Rightarrow \langle s \rangle \langle s \rangle$ the teacher drinks tea $\langle /s \rangle$

Source: DeepLearning.AI, *NLP with Probabilistic Models*, Week 3 slides.

Corpus:

- ▶ `<s> Lyn drinks chocolate </s>`
- ▶ `<s> John drinks tea </s>`
- ▶ `<s> Lyn eats chocolate </s>`

$$P(\text{Lyn} \mid \text{<s>}) = \frac{2}{3}$$

$$P(\text{drinks} \mid \text{Lyn}) = \frac{1}{2}$$

$$P(\text{chocolate} \mid \text{eats}) = \frac{1}{1}$$

$$P(\text{John} \mid \text{<s>}) = \frac{1}{3}$$

$$P(\text{chocolate} \mid \text{drinks}) = \frac{1}{2}$$

$$P(\text{</s>} \mid \text{tea}) = \frac{1}{1}$$

Multiplying along `<s> Lyn drinks chocolate </s>` gives $\frac{2}{3} \times \frac{1}{2} \times \frac{1}{2} \times \frac{2}{2} = \frac{1}{6}$.

Objective: Apply sequence probability approximation with bigrams after adding start and end word.

Question:

Given these conditional probabilities

$$P(\text{Mary})=0.1; \quad P(\text{likes})=0.2; \quad P(\text{cats})=0.3$$

$$P(\text{Mary}|\text{<s>})=0.2; \quad P(\text{</s>}|\text{cats})=0.6$$

$$P(\text{likes}|\text{Mary})=0.3; \quad P(\text{cats}|\text{likes})=0.1$$

Approximate the probability of the following sentence with bigrams: "<s> Mary likes cats </s>"

Type: Multiple Choice, single answer

Options and solution:

1. $P(\text{<s> Mary likes cats </s>}) = 0$

2. $P(\text{<s> Mary likes cats </s>}) = 0.0036$

3. $P(\text{<s> Mary likes cats </s>}) = 0.003$

4. $P(\text{<s> Mary likes cats </s>}) = 1$

Source: DeepLearning.AI, *NLP with Probabilistic Models*, Week 3 slides.

Count Matrix: A matrix that counts occurrences of word pairs in a corpus.

Probability Matrix: A matrix that calculates probabilities of word pairs based on counts.

Example: For the sentence “I love NLP. I love AI.”

- ▶ Count Matrix: Counts how many times each word appears with every other word.
- ▶ Probability Matrix: Calculates the probability of each word appearing given the previous word.

$$P(w_n | w_{n-N+1}^{n-1}) = \frac{C(w_{n-N+1}^{n-1}, w_n)}{C(w_{n-N+1}^{n-1})}$$

- Rows: unique corpus (N-1)-grams
- Columns: unique corpus words

Corpus: `<s>I study I learn</s>`

- Bigram count matrix

"study I" bigram

	<s>	</s>	I	study	learn
<s>	0	0	1	0	0
</s>	0	0	0	0	0
I	0	0	0	1	1
study	0	0	1	0	0
learn	0	1	0	0	0

Source: DeepLearning.AI, *NLP with Probabilistic Models*, Week 3 slides.

Text: "I love NLP. I love AI."

Bigrams:

- ▶ (I, love): 2
- ▶ (love, NLP): 1
- ▶ (love, AI): 1

Count Matrix:

- ▶ Rows: Words in the corpus
- ▶ Columns: Words in the corpus
- ▶ Cells: Count of occurrences of each word pair

$$P(w_n | w_{n-N+1}^{n-1}) = \frac{C(w_{n-N+1}^{n-1}, w_n)}{C(w_{n-N+1}^{n-1})}$$

- Divide each cell by its row sum

Corpus: <s>I study I learn</s>

Count matrix (bigram)

	<s>	</s>	I	study	learn	sum
<s>	0	0	1	0	0	1
</s>	0	0	0	0	0	0
I	0	0	0	1	1	2
study	0	0	1	0	0	1
learn	0	1	0	0	0	1



Probability matrix

	<s>	</s>	I	study	learn
<s>	0	0	1	0	0
</s>	0	0	0	0	0
I	0	0	0	0.5	0.5
study	0	0	1	0	0
learn	0	1	0	0	0

Source: DeepLearning.AI, *NLP with Probabilistic Models*, Week 3 slides.

Probability Calculation:

Bigram Probability:

$$P(\text{love} \mid \text{I}) = \frac{\text{Count}(\text{I}, \text{love})}{\text{Count}(\text{I})} = \frac{2}{2} = 1$$

Probability Matrix:

- ▶ Rows: Words in the corpus
- ▶ Columns: Words in the corpus
- ▶ Cells: Probability of each word given the previous word

A **language model** assigns a probability to any sequence of words. The probability matrix is one: read a row to predict the next word, multiply along a path to score a whole sentence.

- probability matrix $\Rightarrow$ language model
 - Sentence probability
 - Next word prediction

	<s>	</s>	I	study	learn
<s>	0	0	1	0	0
</s>	0	0	0	0	0
I	0	0	0	0.5	0.5
study	0	0	1	0	0
learn	0	1	0	0	0

Sentence probability:

<s> I learn </s>

$$\begin{aligned} P(\text{sentence}) &= \\ P(I|\text{<s>})P(\text{learn}|I)P(\text{</s>}|\text{learn}) &= \\ 1 \times 0.5 \times 1 &= \\ 0.5 \end{aligned}$$

Source: DeepLearning.AI, *NLP with Probabilistic Models*, Week 3 slides.

N-gram Models: **Out of vocabulary words**

Problem: Many words in a language are not present in the training corpus, leading to OOV issues.

Problem: Many words in a language are not present in the training corpus, leading to OOV issues.

Example: The word “quokka” might not be in the training data.

Problem: Many words in a language are not present in the training corpus, leading to OOV issues.

Example: The word “quokka” might not be in the training data.

Impact: OOV words can lead to poor model performance and inaccurate predictions.

Problem: Many words in a language are not present in the training corpus, leading to OOV issues.

Example: The word “quokka” might not be in the training data.

Impact: OOV words can lead to poor model performance and inaccurate predictions.

Closed vs. Open Vocabularies:

- ▶ **Closed vocabulary:** Only words seen during training are recognized.
- ▶ **Open vocabulary:** Model can handle unseen words.

Problem: Many words in a language are not present in the training corpus, leading to OOV issues.

Example: The word “quokka” might not be in the training data.

Impact: OOV words can lead to poor model performance and inaccurate predictions.

Closed vs. Open Vocabularies:

- ▶ **Closed vocabulary:** Only words seen during training are recognized.
- ▶ **Open vocabulary:** Model can handle unseen words.

Solution: Use a special tag <UNK> in the corpus and input to represent unknown words.

Using <UNK>: Replace rare or unseen words with the special token <UNK>.

Using <UNK>: Replace rare or unseen words with the special token <UNK>.

Why? Helps the model generalize to words it has not seen during training.

Using <UNK>: Replace rare or unseen words with the special token <UNK>.

Why? Helps the model generalize to words it has not seen during training.

Example:

- ▶ Original: I love ChatGPT
- ▶ With OOV handling: I love <UNK>

1. **Create vocabulary V :** Build a list of all words to be recognized (e.g., most frequent words).
2. **Replace OOV words:** For any word in the corpus not in V , replace it with <UNK>.
3. **Estimate probabilities:** Treat <UNK> as a regular word when computing word probabilities.

This approach allows the model to handle unseen words gracefully during inference.

Corpus

<s> Lyn drinks chocolate </s>
<s> John drinks tea </s>
<s> Lyn eats chocolate </s>



Corpus

<s> Lyn drinks chocolate </s>
<s> <UNK> drinks <UNK> </s>
<s> Lyn <UNK> chocolate </s>

Min frequency $f=2$

Vocabulary

Lyn, drinks, chocolate

Input query

<s> Adam drinks chocolate </s>



<s> <UNK> drinks chocolate </s>

Source: DeepLearning.AI, *NLP with Probabilistic Models*, Week 3 slides.

Criteria for Vocabulary Selection:

- ▶ **Minimum word frequency f :** Only include words that appear at least f times in the corpus.
- ▶ **Maximum vocabulary size $|V|$:** Limit V to the top $|V|$ most frequent words.

Use $\langle \text{UNK} \rangle$ Sparingly: Choose f and $|V|$ to minimize the number of words replaced by $\langle \text{UNK} \rangle$, while keeping the vocabulary manageable.

Caution: A large $\langle \text{UNK} \rangle$ rate makes a model look good on its own terms, because predicting one frequent token is easy. Two models are only comparable if they were built with the same vocabulary V .

N-gram Models: **Smoothing**

<UNK> fixes words missing from V . It does nothing for a pair of words that are both in V but never appeared *together*.

For the corpus I am happy because I am learning we computed

$$P(\text{happy} \mid \text{I}) = \frac{C(\text{I happy})}{C(\text{I})} = \frac{0}{2} = 0$$

even though “I” and “happy” are both perfectly ordinary words.

- ▶ One zero factor drives the whole sentence probability to 0.
- ▶ $\log 0 = -\infty$, so perplexity is undefined rather than merely bad.
- ▶ Longer contexts make this worse: most trigrams a model meets at test time were never seen in training.

Idea: pretend every possible n-gram was seen once more than it was.

$$P_{\text{Laplace}}(w_i \mid w_{i-1}) = \frac{C(w_{i-1}, w_i) + 1}{C(w_{i-1}) + |V|}$$

- ▶ The +1 in the numerator lifts unseen pairs off zero.
- ▶ The +|V| in the denominator is what keeps each row summing to 1: one unit is added for every word that could follow w_{i-1} .
- ▶ **Add-k** is the same move with a smaller nudge, $k < 1$:

$$P_{\text{add-}k}(w_i \mid w_{i-1}) = \frac{C(w_{i-1}, w_i) + k}{C(w_{i-1}) + k|V|}$$

Worked example. Same corpus, $V = \{\text{I, am, happy, because, learning}\}$, so $|V| = 5$ and $C(\text{I}) = 2$.

Unsmoothed

$$P(\text{am} \mid \text{I}) = \frac{2}{2} = 1$$

$$P(\text{happy} \mid \text{I}) = \frac{0}{2} = 0$$

Add-one

$$P(\text{am} \mid \text{I}) = \frac{2+1}{2+5} = \frac{3}{7}$$

$$P(\text{happy} \mid \text{I}) = \frac{0+1}{2+5} = \frac{1}{7}$$

Nothing is zero any more, and the five smoothed probabilities still sum to 1. The mass came out of the one bigram we actually observed, whose estimate fell from 1 to 3/7.

The trade-off: that is a lot of mass to give away.

- ▶ Real vocabularies hold tens of thousands of words, so $|V|$ dwarfs $C(w_{i-1})$ and add-one hands almost all the probability to n-grams that were never seen.
- ▶ Add- k softens this but leaves the same flaw: every unseen n-gram gets the *same* estimate, whether it is plausible English or nonsense.
- ▶ Practical models instead fall back on shorter contexts, which are estimated from more data. If the trigram (w_{i-2}, w_{i-1}, w_i) is unseen, use the bigram (w_{i-1}, w_i) ; if that is unseen too, use the unigram.
 - **Backoff:** drop to the shorter context only when the longer one has count zero.
 - **Interpolation:** always mix all orders, $\hat{P} = \lambda_1 P(w_i) + \lambda_2 P(w_i | w_{i-1}) + \lambda_3 P(w_i | w_{i-2}, w_{i-1})$, with $\sum_j \lambda_j = 1$ and the λ_j tuned on held-out data.

- ▶ Perplexity (PPL) measures how well a language model predicts a sequence of words. Intuitively, it reflects “on average, how many choices the model is unsure about” per word.
- ▶ Formula:

$$\text{PPL}(W) = P(w_1, \dots, w_N)^{-\frac{1}{N}} = \exp \left(-\frac{1}{N} \sum_{i=1}^N \log P(w_i \mid w_1, \dots, w_{i-1}) \right)$$

- ▶ How to interpret:
 - Lower perplexity $\rightarrow$ model assigns higher probability to the test set sequences $\rightarrow$ better predictions.

- ▶ Why it matters:
 - Standard metric for evaluating language models, from n-gram models to neural LMs.
 - Only compare perplexities of models that share the same vocabulary V .
- ▶ Example, scoring both models on the same held-out text with the same V :
 - Bigram model $\rightarrow$ PPL ≈ 200 : on average it is choosing between roughly two hundred plausible next words.
 - Neural LM $\rightarrow$ PPL ≈ 20 : the same choice, narrowed tenfold.
- ▶ Caveat: perplexity rewards fluency on held-out text, and nothing else. It says nothing about whether the model is truthful, useful, or safe.

N-gram Models: **Limitations**

- ⚠ **Data Sparsity**
- ⚠ **Limited Context** (only $N-1$ words)
- ⚠ **Explodes with Vocabulary Size**
- ⚠ **Doesn't Capture Semantics/Syntax**

Beyond N-grams, NLP as a whole still faces some big challenges:

- ▶ **Bias:** Models can pick up and even increase unfairness from the data they learn from.
- ▶ **Needs Lots of Data:** Modern NLP needs huge amounts of labeled text, which is hard and expensive to get.
- ▶ **Ambiguity:** Language can be confusing. In “I saw the man with the telescope”, who has the telescope?
- ▶ **Many Languages:** It’s tough to make models that work well for every language, especially those with little data.
- ▶ **Expensive to Train:** Training big models costs a lot of money and energy, so not everyone can do it.

N-gram Models: **Summary**

- ▶ Word Embeddings (Word2Vec) and Neural Language Models (RNNs, LSTMs)
- ▶ Transformer-based Models (BERT, GPT)
- ▶ Subword Tokenization (Byte-Pair Encoding)
- ▶ Pretrained Language Models
- ▶ Contextual Representations

- ▶ NLP enables machines to understand human language
- ▶ N-grams model word sequences simply and effectively
- ▶ Sequence notation and probability estimation are essential
- ▶ Start/end/UNK tokens improve modeling and robustness
- ▶ Smoothing keeps unseen contexts from collapsing a sentence to probability zero
- ▶ Perplexity scores a model on held-out text, given a fixed vocabulary
- ▶ N-gram models are limited → neural models offer solutions

NLP: References

- ▶ Jurafsky, D., & Martin, J. H. (2026). *Speech and Language Processing (3rd Ed Draft)* – <https://web.stanford.edu/~jurafsky/slp3/>
- ▶ CMU CS 11-711 Advanced NLP (Graham Neubig) – <https://cmu-anlp.github.io/>
- ▶ Stanford CS224N, Natural Language Processing with Deep Learning – <https://web.stanford.edu/class/cs224n/>
- ▶ Manning, C. D., Raghavan, P., & Schütze, H. (2008). *Introduction to Information Retrieval* – <https://nlp.stanford.edu/IR-book/>
- ▶ Bengio et al. (2003) – A Neural Probabilistic Language Model, JMLR 3, 1137–1155
- ▶ Mikolov et al. (2013) – Efficient Estimation of Word Representations in Vector Space
- ▶ Vaswani et al. (2017) – Attention Is All You Need
- ▶ Younes Bensouda Mourri & Łukasz Kaiser, *Natural Language Processing Specialization*, DeepLearning.AI – <https://www.deeplearning.ai/specializations/natural-language-processing>
- ▶ Dan Klein, *Natural Language Processing* slides, UC Berkeley (via T. Berg-Kirkpatrick, CMU) – https://www.cs.cmu.edu/~arielpro/15381f16/slides/NLP_guest_lecture.pdf